Use cases, portals and SQL activity

A timed 10-minute walkthrough + technical reference

Scope: the demonstrated **Next.js + SQLite application**. Explains every administrator tab, the student portal, how updates travel through the system, and how UI commands become SQL.

Presentation plan — exactly 10 minutes

Time	Show / explain
0:00–1:00	Purpose, roles, technology and architecture.
1:00–3:30	Administrator workspace: all sidebar tabs and the academic workflow.
3:30–5:00	Perform an update; explain validation, persistence and refresh.
5:00–7:00	Student portal: all four tabs and access restrictions.
7:00–9:00	SQL activity: columns, examples and limitations.
9:00–10:00	Reports, snapshot, integrity, verification and conclusion.

The numbered speaking sections total 10 minutes. Later pages are reference material for questions, not additional speech. Rehearse the clicks beforehand; avoid creating unnecessary academic records during the presentation.

Preparation checklist

- Run `npm run dev` from the repository root; open `http://localhost:3000`.
- Use two browser profiles or an incognito window: one admin, one student. Two ordinary tabs share the same session cookie.
- Find the student's existing enrollment ID before adding attendance. Choose a date with no attendance record for that enrollment.
- Keep SQL activity open in the admin profile. Be ready to pause polling after capturing an operation.
- Show the actual data present; seeded sample records are persisted demonstration data, not live university records.

Editable source: learnit/usecase.html • PDF: learnit/usecase.pdf • Source reference: web/src and web/lib.

SPEAKING SECTION 1 / 0:00–1:00

Purpose, roles and technology

> “CampusOS connects college administration with a student’s academic view. Administrators maintain records; students only read information linked to their account. My focus is the frontend, backend integration and codebase management. I will demonstrate an administrative update, show its student-facing result and explain the SQL behind it.”

Technology	Responsibility
Next.js 15 + React 19	App Router hosts the application and server API routes. React components render screens and update UI state.
Tailwind CSS 3 + Lucide	Reusable utility styling, responsive layouts, icons, charcoal surfaces and green accent.
Axios + React Hook Form	HTTP requests and form management/validation. Recharts renders dashboard charts.
Node.js + node:sqlite	Server-side JavaScript executes parameterized SQL directly; no ORM in this build.
SQLite	Persistent local database, foreign keys enabled, WAL mode, constraints and triggers.
Node tests + Puppeteer	API tests and real Chromium browser checks.

> “The browser sends an HTTP request to a Next.js API route. The server checks the session and role, validates input, executes SQL and returns JSON. React then renders the result. The browser never opens the database directly.”

```
React UI → Axios → /api/[...path] → portal.handle()
         → session / role checks → academic or portal service
         → node:sqlite → campus.sqlite → JSON → React
```

There is also a separate React/Vite + FastAPI + PostgreSQL build in the repository. It is an independent runtime, not a shared live database. The portal and SQL viewer described here belong to the Next.js/SQLite build. The academic REST interfaces are similar; the extended features are not identical.

Administrator tabs: academic records

> “The admin workspace follows the actual academic workflow: set up departments and courses, schedule course offerings in semesters, assign faculty, enroll students and record grades. Reusable management screens keep CRUD behavior consistent.”

Sidebar tab	What it shows / what admin can do
Dashboard	Six summary cards: students, faculty, departments, courses, active enrollments and active semesters. Charts show department distribution, enrollment statuses, admissions by year and offerings by semester. Recent students/enrollments, popular courses and department counts summarize activity.
Students	Student ID, enrollment identity, personal/contact information, department, admission year and status. Create, edit, search/filter, paginate, delete where dependencies allow; open a student detail page with enrollments.
Faculty	Faculty ID, employee identity, contact details, designation and department. CRUD and detail view showing teaching assignments.
Departments	Department ID, code, name, office and related student/faculty/course counts. Maintain academic units. Dependent academic records restrict deletion.
Courses	Course ID, code, title, credits, department and description. Maintain the catalog; course details show its offered instances.
Semesters	Semester ID, name, academic year, date range, status and offering count. Maintain academic periods; details show scheduled offerings.
Course Offerings	Offering ID, course, semester, section, capacity, occupied/free seats and assigned faculty. Create the specific instance of a catalog course. Details show enrolled students and faculty.
Faculty Assignments	Assignment ID linking a faculty member to an offering, with joined course, semester and section information. Add/remove teaching allocations.
Enrollments	Enrollment ID linking a student to an offering, with date, status and joined academic/grade information. Register a student or update status, including Dropped.
Grades	Grade ID, linked enrollment/student/course, marks, maximum marks, letter and percentage. Add/edit assessment results. Marks cannot exceed the maximum.

Academic list pages support configured search, filters, sort and pagination. The primary key identifies a row; readable labels come from SQL joins. A course is the catalog definition; an offering is that course taught in a particular semester and section.

Administrator tabs: operations and tools

Sidebar tab	What it shows / what admin can do
Reports	Three sub-tabs: Student reports (department, admission year, status); Faculty reports (department counts, teaching assignments); Academic reports (course enrollment, capacity utilization, semester summary, individual grade rows). Export displayed tables to CSV; a server CSV link exports students by department. This screen does not implement GPA.
Attendance	Attendance ID, enrollment ID, date and Present / Absent / Late / Excused status. CRUD per enrollment/date; that pair must be unique. A student sees only records belonging to their enrollments.
Library books	Book ID, ISBN, title, author and copy count. Maintain the catalog. ISBN is unique; copies must be positive.
Library loans	Loan ID, student ID, book ID, borrowed date, due date and optional returned date. Issue/update a loan; record a return by setting returned_date. Dates are validated. There is no automatic fine or stock-availability enforcement.
Student access	Provision a student account using an existing student ID and a password of 12–256 characters. Login email comes from that student's database record; it is not freely selected. This is provisioning, not a password-reset/account-edit console.
SQL activity	Read-only recent SQL templates and request outcomes, refreshed every three seconds. Pause/resume polling or refresh manually. It does not accept arbitrary SQL commands.
Settings	Database/API health, local SQLite file path, theme toggle, summary counts, access explanation, documentation links and current SQL snapshot download. Theme preferences are browser-local.

> "Attendance and library records are maintained by administrators but displayed read-only to students. Student access creates the account-to-student link. SQL activity is my explanation tool: it connects a UI operation to the query the server actually executed."

Attendance and library endpoints currently implement paginated lists ordered by descending ID; the generic UI's search/sort controls do not provide full server-side search/sort for these three new resources. Do not promise every tab has identical filtering.

Permission boundary

The server authorizes operations, not just the sidebar. Unauthenticated protected requests return **401**. A student requesting an administrator resource returns **403**. An admin can manage domain data but is not given a free-form database console.

SPEAKING SECTION 3 / 3:30–5:00

Demonstrate an update end to end

Recommended live action: attendance correction

1. In admin Attendance, select an existing row belonging to the demonstration student. Note its enrollment ID and date.
2. Edit its status, for example Absent → Present, and save.
3. Show the success message and refreshed admin list.
4. Open SQL activity: look for `PUT /api/attendance/<id>` and its SELECT → UPDATE → SELECT sequence.
5. Reload the separate student window, then open Attendance. The corrected status and percentage reflect the persisted change.

> “A form change is only React state until I click Save. The browser submits JSON using PUT. The backend checks my role, validates the status, checks the row and executes a parameterized UPDATE. SQLite persists the new value, the server returns the updated row, and the admin screen refetches its list. The student’s next data load reads the same database, so both views stay consistent without duplicating the academic record.”

How the frontend is built

- `CollegeApp.jsx`: checks the session, chooses Login, admin workspace or student portal, and maps URL paths to screens.
- `ResourcePage.jsx` + `config/resources.js`: one CRUD page receives entity-specific columns, forms, endpoints and warnings.
- Shared DataTable, FormModal, Field, Pagination, confirmation dialogs, loading skeletons, errors and toast messages avoid duplicate UI logic.
- After successful CRUD, the page invalidates cached dropdown options and refetches its current list.

What “updated everywhere” actually means

Records are persisted immediately, but ordinary screens are **request-driven, not WebSocket live**. Other admin pages refresh on reload/re-entry. The student portal loads all four datasets on mount or retry; switching its tabs only changes the visible panel and does not refetch. Reload it after admin changes. Only SQL activity automatically polls.

Core academic CRUD uses explicit BEGIN IMMEDIATE / COMMIT / ROLLBACK transactions. Attendance/library writes use SQLite’s per-statement atomic writes, then read back the row and refresh the snapshot. Do not describe all handlers as one identical transaction implementation.

SPEAKING SECTION 4 / 5:00–7:00

Student portal: all four tabs

> “The student portal is deliberately view-only. Students cannot change marks, enroll themselves, mark attendance or issue books. The session contains the account identity; the server derives its linked student ID and uses that ID in every portal query. Supplying somebody else’s ID in a URL does not change the authorized student.”

Portal tab	Exactly what is displayed
Overview	Cards for number of course enrollment rows, sum of their credits, attendance percentage and active loans. Profile: full name, enrollment number, email, department name/code. Library glance: unreturned book count and the earliest due date among outstanding loans.
My courses	Course name/code, semester and academic year, section, credits, grade letter and marks obtained / maximum. A LEFT JOIN keeps an enrollment visible when it has no grade, displaying a dash.
Attendance	Overall rate and Present count / total records. Rows show date, course code/name and status. Records are ordered newest date first.
Library	Borrowed title, author, ISBN, borrowed/due/returned dates and derived Returned / Overdue / On loan status. This is the student’s own loan history, not a searchable whole-library catalog.

Exact calculation rules — useful viva questions

- **Attendance:** round(Present records ÷ all attendance records × 100). Late and Excused are included in the denominator but not counted Present. No records displays a dash.
- **Course cards:** current code counts all returned enrollment rows and sums their credits, without excluding Dropped or Completed. Explain it as a summary of stored enrollment records, not guaranteed active registration.
- **Active loans:** rows with no returned_date. Overdue means unreturned and past the due date; the status is derived in the browser rather than stored in a status column.
- **Refresh behavior:** four GET requests load in parallel. Tab switches use the fetched data. Reload after administrator updates.

Student API calls

```
GET /api/portal/me          → profile + department
GET /api/portal/courses     → own enrollments + courses + grades
GET /api/portal/attendance  → own attendance + course labels
GET /api/portal/library     → own loans + book details
```

Sign out revokes the current session on the server and clears its cookie. Student permission is enforced even if someone bypasses the UI and calls an admin endpoint directly.

SPEAKING SECTION 5 / 7:00–9:00

Reading the SQL activity log

> “This is an application-level SQL activity viewer, not a database editor. It captures the templates passed to the application’s SQLite methods. Each row connects a query to an HTTP request, so I can show what ran when I opened a page or saved a record. Parameters are deliberately not stored, so private form values and credentials are not printed.”

Visible column	Meaning
Time	Timestamp of the captured statement, displayed in local time.
Result	Success / Failed / Pending from the overall HTTP handler outcome. A successful SELECT inside a failed request is also labelled Failed. This is not proof that each individual statement failed or committed.
Duration	Measured statement execution time in milliseconds. Not total network or page-load latency.
Request	HTTP method + API path, plus request UUID. Several statements with the same UUID belong to one operation. Query-string filters are not displayed in the recorded path.
Role	Role associated with the request, such as admin or student.
SQL template	SQL with bound values omitted; quoted string literals and numeric literals are also masked to ?. Whitespace/aliases can vary from simplified examples in this guide.
Rows	For all(): returned row count; get(): 1 or 0; run(): changed rows. Statements without a count may display 0 because null is currently coerced to a number. It is not a universal “rows scanned” metric.

What the viewer captures — and excludes

- Wraps application calls to prepared statements’ all/get/run and database exec. It does **not** separately trace SQL executed internally inside a trigger.
- Last 200 events, newest shown first; in-memory per server process. Restart clears history; heavy snapshot/export reads can replace older events.
- Polls `GET /api/dbviewer` every three seconds. Pausing stops polling, not SQL execution or event collection. Refresh fetches the current buffer.
- Queries containing account/session/login_throttle are excluded. Session lookup happens before the audit context; schema initialization is also outside it. Thus login/logout does not expose credential SQL.
- Authorization rejected before the context is entered normally produces no SQL audit event. The viewer fetch itself does not execute a domain query and normally adds no event.

The log is an observability aid, not a durable or tamper-proof compliance trail. It does not show parameter values, SQLite query plans, or every statement run by external tools.

SPEAKING SECTION 6 / 9:00–10:00

Reports, persistence and closing

> “Reports turn academic records into useful summaries: student distribution, faculty workload, course enrollment, capacity utilization, semester totals and grade rows. CSV exports make them portable. Some calculations use SQL GROUP BY and COUNT; others combine joined SQL results in JavaScript. I do not claim every chart is one SQL aggregate.”

SQL snapshot: a reproducible domain-data copy

- Normal writes update the persistent file at `web/lib/.data/campus.sqlite`.
- Successful domain writes in the default local database also refresh `database/live_snapshot.sql`. This is a full SQL snapshot, not an append-only history of commands.
- Settings → Download current database as SQL calls `GET /api/database/export.sql`. It generates the current domain schema and rows, excluding accounts, sessions, throttles and migration markers.
- Snapshot generation performs schema metadata reads, PRAGMA table_info and SELECT * for domain tables; these can appear in the activity log under the triggering request.
- Core academic snapshot refresh failure is logged after commit, without undoing the committed data. Portal CRUD can report an error if its post-write snapshot refresh fails, even though the row write succeeded. Check before manually retrying a mutation.

> “Integrity is enforced below the UI: foreign keys protect relationships, unique constraints stop duplicates, check constraints reject invalid values and capacity triggers reject over-enrollment. Tests cover APIs, constraints, authorization and the query audit; a browser smoke test checks the portal flow. The result is a working local administrative demo: one source of domain data, different role-specific views, and a visible explanation of the SQL behind each action.”

Security and reliability — keep claims precise

Passwords are salted scrypt hashes; the session cookie holds a random token whose SHA-256 hash is stored server-side. Cookies are HttpOnly and SameSite=Strict, with Secure in production; sessions last eight hours. Login throttling blocks after five counted attempts in a fifteen-minute window and resets on success. Cross-origin mutation checks supplement role checks. Only GET requests retry automatically once; mutations are not silently replayed.

Production requires HTTPS, changed demo credentials, backup/restore testing, deployment-level rate limiting and access review. SQLite is persistent but node:sqlite calls are synchronous; this design is for a local/small demo, not a claim of unlimited concurrent scale.

What runs on which command?

Templates below follow the implementation. Column order and joined SELECT text vary by resource and supplied fields. The request may produce additional reads for option lists, response details, the refreshed list or the SQL snapshot.

UI command	Request and SQL behavior
Open an academic list	GET /api/students (or another resource). Runs the resource's joined SELECT. A LIMIT 0 query inspects result columns for allowed sorting. Search, filters, sorting and page slicing happen in Node for these nine resource lists; do not expect a WHERE search or LIMIT page clause for every click.
Type search / change filter / sort / page	New GET with query parameters after state/debounce changes. The joined SQL template can stay identical because filtering/pagination occurs after fetching rows in Node.
Open an Add/Edit form	May GET /api/departments/options, /students/options, /offerings/options, etc. These read joined resource data and map it to dropdown labels, capped at 500 options. Cached options may avoid another request.
Create academic record	POST /api/resource. BEGIN IMMEDIATE → INSERT with ? placeholders → joined detail SELECT → COMMIT. On failure: ROLLBACK. Then snapshot reads; frontend GET refetch follows.
Edit academic record	PUT /api/resource/id. BEGIN IMMEDIATE → SELECT old row WHERE pk=? → UPDATE supplied fields WHERE pk=? → joined detail SELECT → COMMIT. Validation failure rolls back; snapshot follows success.
Confirm Delete	DELETE /api/resource/id. BEGIN IMMEDIATE → SELECT old row → DELETE WHERE pk=? → COMMIT. Dependent FK rows cause a readable conflict and ROLLBACK. Cancelling confirmation sends no DELETE.
Open student/course/etc. detail	GET /api/resource/id. SELECT * FROM (resource joined query) WHERE pk=?. Extra joined reads load related enrollments, offerings, assignments or students according to the detail screen.
Attendance/library list	GET /api/attendance, /library-books or /library-loans. SELECT count(*) → SELECT * ORDER BY primary_key DESC LIMIT ? OFFSET ?. These endpoints paginate inside SQL.
Attendance/library create/edit/delete	POST / PUT / DELETE corresponding resource. Edit/delete first SELECT the existing row. INSERT or UPDATE reads the resulting row back; DELETE removes it. SQLite atomic statement writes; snapshot after success.

CRUD and integrity examples

1. Update an attendance status

```
-- PUT /api/attendance/42 (42 is illustrative)
SELECT * FROM attendance WHERE attendance_id=?;
UPDATE attendance SET status=? WHERE attendance_id=?;
SELECT * FROM attendance WHERE attendance_id=?;
```

The bound values could be Present and the selected ID; the viewer does not show them. Depending on the form payload, UPDATE may include enrollment_id and date as well. Snapshot reads follow, then a separate GET refreshes the admin list.

2. Create a student

```
-- POST /api/students (representative field subset)
BEGIN IMMEDIATE;
INSERT INTO student
(enrollment_number,first_name,last_name,email,admission_year,status)
VALUES (?,?,?,?,?,?);
-- Joined detail SELECT using the generated student_id
COMMIT;
```

A duplicate email raises a unique-constraint conflict: the transaction rolls back and the frontend displays a readable 409. Backend validation may reject input before INSERT, so an invalid action need not produce a write in the log.

3. Add an enrollment or change its status

```
BEGIN IMMEDIATE;
INSERT INTO enrollment
(student_id,offering_id,enrollment_date,enrollment_status)
VALUES (?,?,?,?);
-- Joined detail SELECT on success
COMMIT;
-- On capacity/uniqueness/FK failure instead: ROLLBACK
```

The capacity trigger checks non-Dropped enrollments against offering capacity. Completed rows also count as occupied under the current rule. UPDATE handles reactivation or moving an enrollment; reducing capacity below occupied seats is blocked. Trigger bodies are enforced but not individually printed by this application wrapper.

4. Issue and return a library book

```
INSERT INTO library_loan
(student_id,book_id,borrowed_date,due_date,returned_date)
VALUES (?,?,?,?,?);
UPDATE library_loan SET returned_date=? WHERE loan_id=?;
```

A return changes the row; the student UI derives Returned from returned_date. The database does not automatically decrement book copies, calculate fines or prevent loans exceeding stock.

Portal queries and derived results

Profile — GET /api/portal/me

```
SELECT s.*, d.department_name, d.department_code
FROM student s LEFT JOIN department d USING(department_id)
WHERE student_id=?;
```

My courses — GET /api/portal/courses

```
SELECT e.*, c.course_code,c.course_name,c.credits,o.section,
       sem.semester_name,sem.academic_year,
       g.marks_obtained,g.max_marks,g.grade_letter
FROM enrollment e
JOIN course_offering o USING(offering_id)
JOIN course c USING(course_id)
JOIN semester sem USING(semester_id)
LEFT JOIN grade g USING(enrollment_id)
WHERE e.student_id=? ORDER BY e.enrollment_id;
```

Attendance — GET /api/portal/attendance

```
SELECT a.*,c.course_code,c.course_name
FROM attendance a JOIN enrollment e USING(enrollment_id)
JOIN course_offering o USING(offering_id)
JOIN course c USING(course_id)
WHERE e.student_id=? ORDER BY a.date DESC,a.attendance_id DESC;
```

Library — GET /api/portal/library

```
SELECT l.*,b.isbn,b.title,b.author
FROM library_loan l JOIN library_book b USING(book_id)
WHERE l.student_id=? ORDER BY l.loan_id DESC;
```

The ? student ID comes from the authenticated account, not an untrusted URL parameter. The browser computes the attendance percentage, credit sum and loan display status from these responses. Switching student tabs does not run a fresh query.

Other commands and what to expect

Command / request	Database or UI work
Open Dashboard	GET /api/dashboard/stats, /charts and /activity?limit=5. Counts, joined resource reads and grouping. Recent records are selected by ID ordering in Node, not a historical change feed.
Report: admission year / status	SELECT admission_year,count(*) students FROM student GROUP BY admission_year ORDER BY admission_year; and SELECT status,count(*) students FROM student GROUP BY status ORDER BY status.
Report: capacity utilization	Joined offerings plus non-Dropped enrollment counts; JavaScript computes round(enrolled × 1000 / capacity) / 10. No separate stored utilization column.
Report: semester summary	Read semesters/enrollments; JavaScript calculates totals and distinct student IDs. Grade report reads joined grade data; percentage is calculated in its SQL projection.
Export table CSV	Table-level CSV uses already-loaded data in the browser, so it need not add SQL. The server CSV link GET /api/reports/export?report=students-by-department runs report reads.
Provision student login	POST /api/admin/accounts: SELECT student email by ID; scrypt password derivation; INSERT portal_account. The account INSERT is excluded from audit. The student email lookup can still appear.
Login / restore session / logout	Server reads/writes account, throttle and session tables; auth statements are excluded. GET /auth/me returns the authenticated account object. Logout deletes the token hash and expires the cookie.
Settings health	GET /api/health executes PRAGMA foreign_keys; /dashboard/stats loads counts. Theme toggle, sidebar collapse and student tab switches change browser state, not domain rows.
Download SQL snapshot	GET /api/database/export.sql reads sqlite_master, PRAGMA table_info and domain SELECT * rows. It generates CREATE/INSERT/index/trigger text for a fresh empty SQLite database; generated INSERT text is not executed during export.

Teacher asks: “Why does the log show many SELECTs after one edit?”

One UI action can include write validation, response joins, snapshot generation and a subsequent list refresh. Group events by request UUID and HTTP path. Snapshot generation can consume much of the 200-event buffer; pause polling after inspecting it, but remember collection continues.

Teacher asks: “Can I type an SQL command here?”

No. This is a read-only activity viewer for application queries, not DBeaver or an arbitrary SQL runner. Limiting it avoids an unrestricted administrative SQL execution surface.

Closing recovery line

“Let me demonstrate that in the application and connect it to the API request and SQL template.” Use the appendix rather than claiming functionality the current build does not implement.

Project: github.com/Bhanu4417/DBMS-PROJCT • Implementation references: web/lib/api.mjs, portal.mjs, queries.mjs, reports.mjs, sql-audit.mjs; web/src/screens/Portal.jsx, DbViewer.jsx, Reports.jsx and components/ResourcePage.jsx.